

- 50 students $\Leftarrow$
- 10 subject for each student $\Leftarrow$

Fundamentals of Programming

CS-114

Week - 10

Arrays



Arrays

- An array is a collection of variables of similar type, placed contiguously in memory. 
- Arrays are used to avoid the inconvenience of giving each data element a unique variable name.
- One data structure having more than one data members of the same type



Array Definition

- Array declaration tells the compiler about

- the type of array. *← → Data type*
- the name of array. *→ name*
- the size of array. *→ # of elements*

type ***arrayName*** ***[arraySize]***; *int NoOfMonths [12];*

↑ *↑* *↑* *↑*

- Example:

float arr [20];

↑ *↑* *↑*

Array type Array name Array size

int NoOfMonth(12);

NoOfMonths

40	9	7	91	20	91	39	7	6	7	7	19
----	---	---	----	----	----	----	---	---	---	---	----

Index

0 1 2 3 4 5 6 7 8 9 10 11

June

Size-1

array

12 diff in tejer

↓ element

index

NoOfMonths { };

NoOfMonths[5]; // 91

Referring to Individual Elements in an Array

- All the array elements are numbered, starting at 0 -> arr[0], arr[1], arr[2], etc

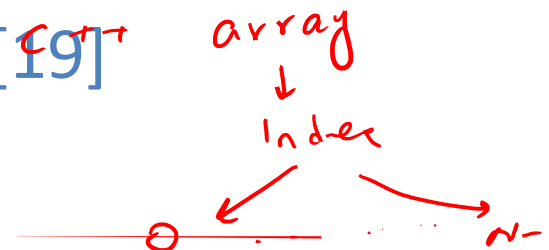
int NoOfMonths[12];
↑ ↑ ↑ ↑

NoOfMonths[3];

- The last array element is one less than the size of the array.

– If array size is 20, ie. arr[**20**] array subscripts would be from

arr[0] to arr[19]

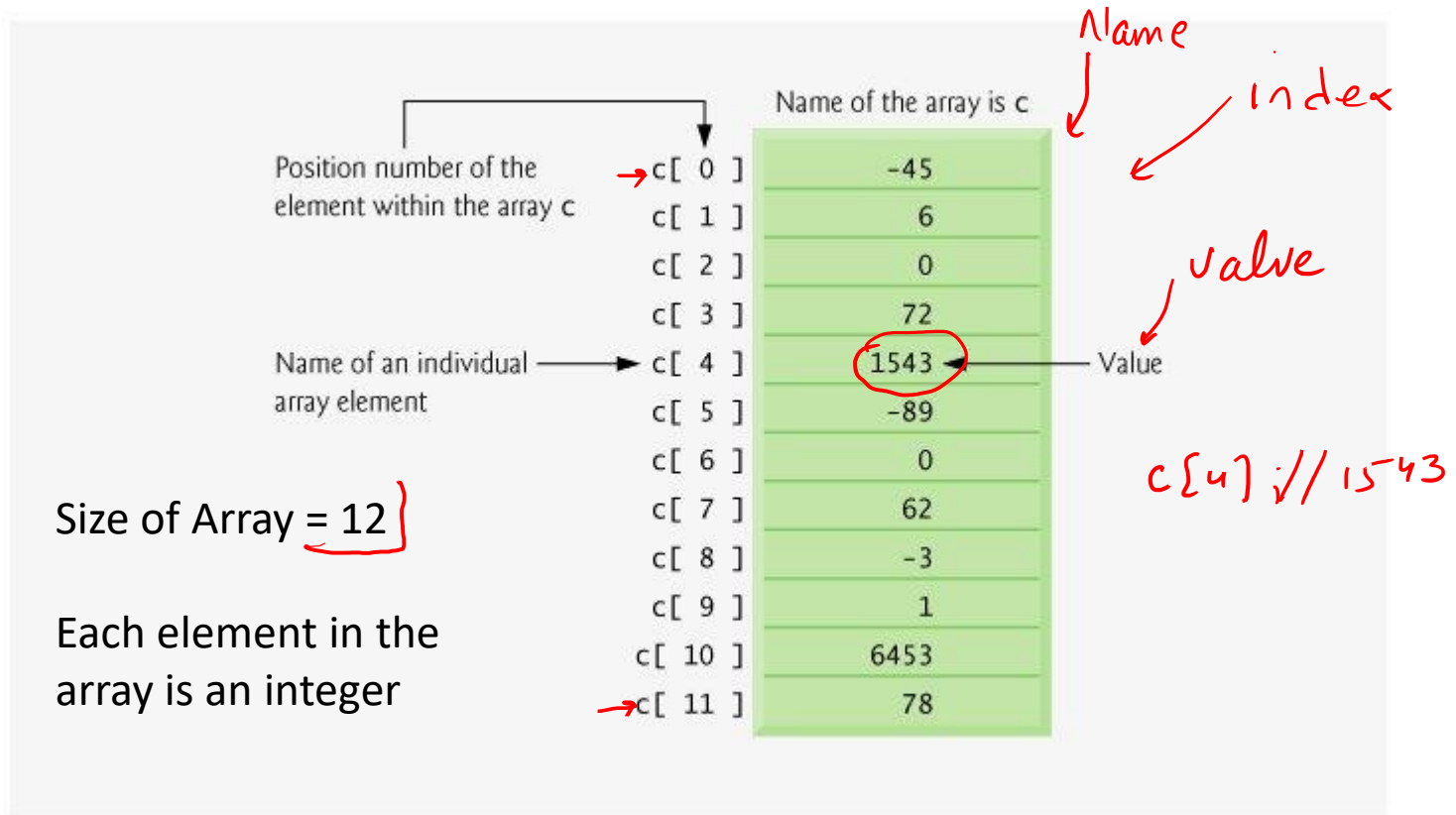


Referring to Individual Elements in an Array

- Individual elements of the array are referred to with subscripts, the number in brackets following the array name `int arr[20];`
 - To use the 2nd element of the array -> `arr[1]`
 - To use the 1st element of the array -> `arr[0]`
 - To use the 9th element of the array -> `arr[8]`

Example

Integer Type Array



(Deitel and Deitel)

`int rooms[5] = { 50, 49, 20, 1, 17 };`

Initializing Arrays

rooms	
0	50
1	49
2	20
3	1
4	17

- `/* integer type array */`

`int rooms[5] = { 50, 49, 20, 1, 17};` ←

- `/* float type array */`

`float length[4] = { 12.39, 16.5,
 15.3, 0.23};`

- `/* character type array */`

`char name[4] = { 'a', 'A', 'z', 'Z'};`

array → 1 data type

Initializing Arrays

- /*declared but uninitialized

```
int n[10]; ←
```

- /* initialize elements of array n with 0

```
int n[ 10 ] = { };
```

```
int n[ 10 ] = { 0 };
```

- /*creates a five-element array.

```
int n[ ] = { 1, 2, 3, 4, 5 };
```

↑

Name	Q ₁	Q ₂	Q ₃
Bilal	7.0	8.0	9.0

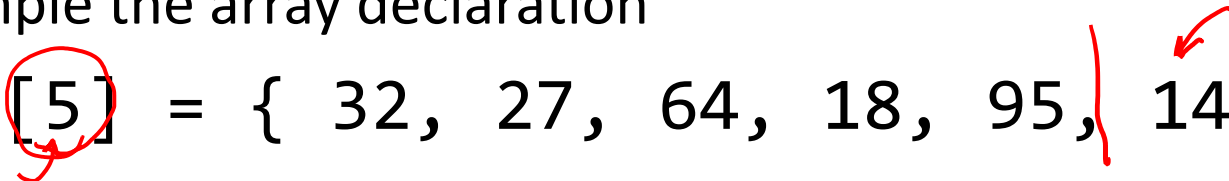
→ structures

Initializing Arrays

- If the array size and an initializer list are specified in an array declaration and the number of initializers are greater than the array size.

- For example the array declaration

`int n[5] = { 32, 27, 64, 18, 95, 14 };`

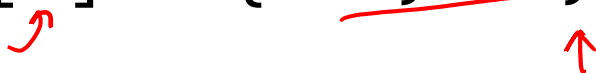


- causes a compilation error, because there are six initializers and only five array elements.

Initializing Arrays

- If the array size and an initializer list are specified in an array declaration and the number of initializers is lesser than the array size.
- For example the array declaration

```
int n[5] = { 32, 27, 64 };
```



{ 32, 27, 64, 0, 0 };

it will fill the remaining array positions with 0

32	27	64	0	0
----	----	----	---	---

Entering Data into an Array

- Data can be entered into individual elements of an array using a loop
- Enter marks of five students into an array

```
int marks[5];  
for ( int i = 0; i < 5; i++ )  
{  
    cout << "Enter marks of " <<i+1 <<" student = ";  
    cin >> marks [i] ;  
}
```

Reading Data from an Array

Reading data from the array “marks” and calculating the average.

```
sum = 0;  
for ( i = 0; i < 5; i++ )  
    sum += marks [i];  
cout<<"Average is "<<sum/5<<endl;
```

Summing the Elements of an Array

```
int main()
{
    → const int Size = 10; //variable indicating
                           size of array
    int a[Size] = {87,68,94,100,83,78,85,91,76,87};
    int total = 0;
    // sum contents of array a
    for ( int i = 0; i < Size; i++ )
        total += a[ i ];

    cout << "Total of array elements: " << total <<
    endl;
    return 0;
}
```

Total of array elements: 849

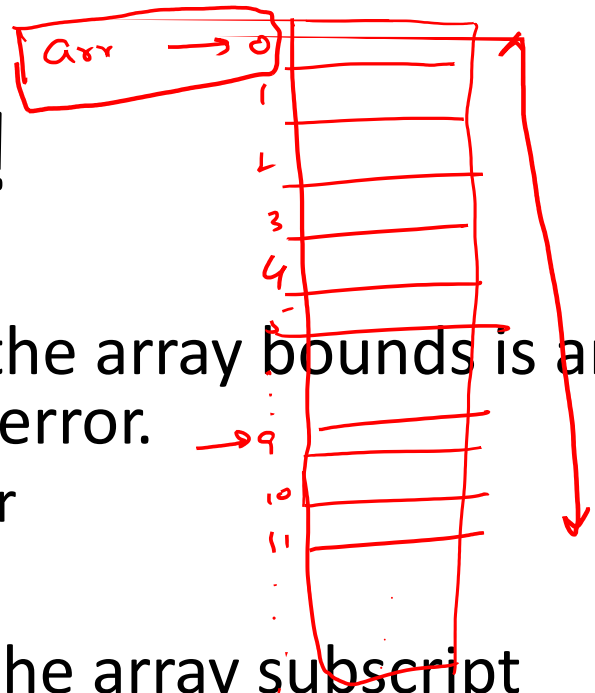
Const variable

- Constant variables must be initialized with a constant value (or expression) when they are declared and cannot be modified thereafter.
- Constant variables are also called **read-only variables**.
- Only constant variables can be used to declare the size of arrays.

```
int arr[10]; // 20
for (int i = 0; i < 20; i++)
    cout << arr[i];
```

Caution!

- Referring to an element outside the array bounds is an **semantic error**. It is not a syntax error.
 - i.e. it is not caught by the compiler
- When looping through an array, the array subscript should never go **below 0** and should always be **less than the total number of elements in the array** (one less than the size of the array).
- Make sure that the loop-termination condition prevents accessing elements outside this range.



Example

```
// Program to copy the contents of one array to another
int main()
{
    int oldMarks[4] = {78, 92, 56, 67};
    int newMarks[4];

    for(int i=0; i<4; i++)
        newMarks[i] = oldMarks[i];

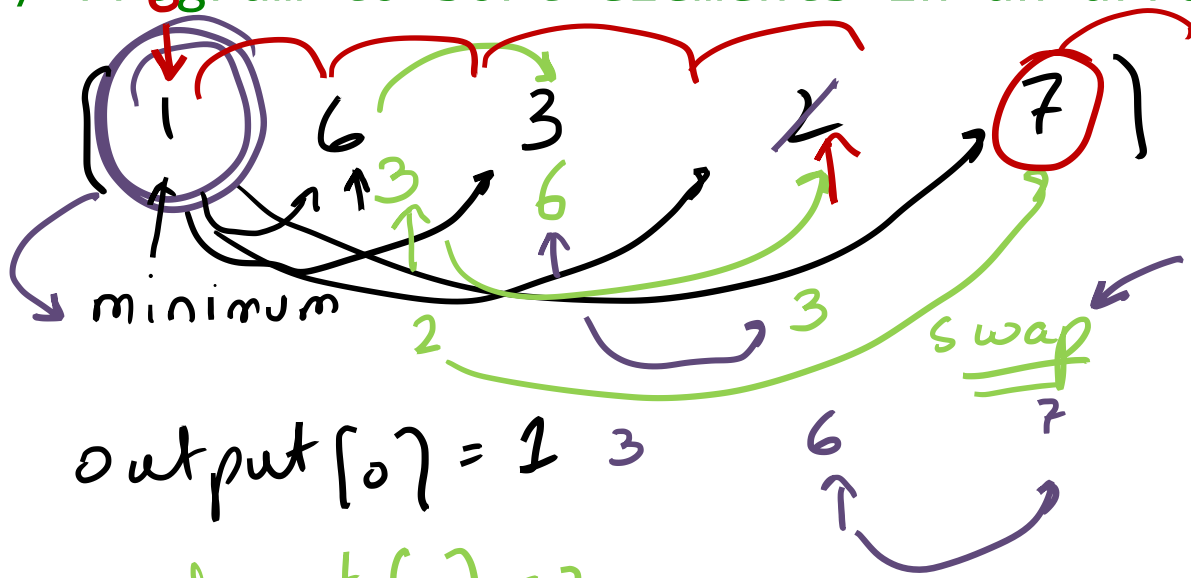
    cout<< "New array is : "<< endl;

    for(int i=0; i<4; i++)
        cout<<"new Marks ["<<i<<" ] : "<<newMarks[i]<<endl;

    return 0;
}
```

Example

// Program to sort elements in an array



output[0] = 1 3

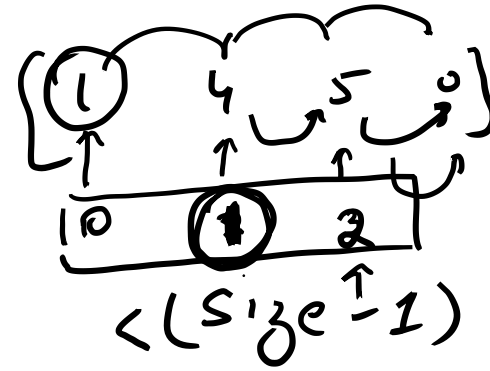
output[1] = 2

output[2] = 3

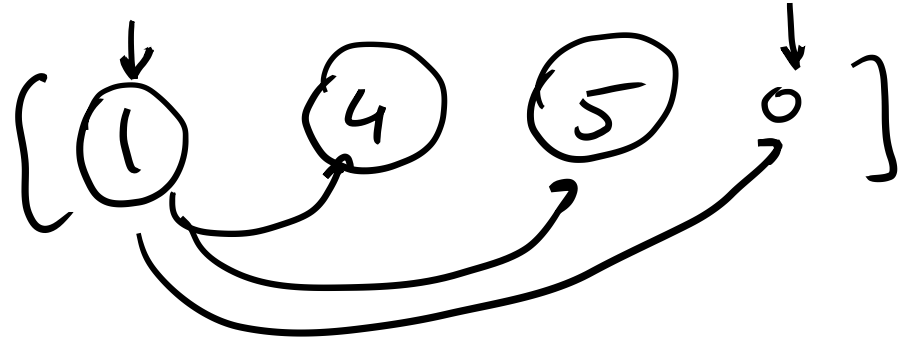
output[3] = 6

output[4] = 7

size = 4



size = 4 $\frac{size}{2}$



$[0 \quad 9 \quad 2 \quad 3]$

iteration = 0

First = 0 $\Rightarrow$ compare

array size = 4

Comparison = 3

at max = size - 1

iterations = $\boxed{\text{size} - 1}$

$i = 0$ $[0 \quad 9 \quad 2 \quad 3]$

$i = 1$

$[0 \quad 2 \quad 9 \quad 3]$

$i = 2$

$[0 \quad 2 \quad 3 \quad 9]$

outer loop ($i = 0, i < \text{size}$)



inner loop ($j = i + 1, j < \text{size} - i$)



if (j value < i value)

$\Rightarrow$ swap